# People of the Holocaust

## Software Requirements Specification

COSC 4920 - Principles of Design

Marquette University

Prepared by:

Version 1.0

October 4, 2026

# Contents

# 1 Revision History

| Version | Date | Author | Description |
|---|---|---|---|
| 1.0 | October 4, 2026 | Leyton Meadows | Initial software requirements specification. |

# 2 Introduction

## 2.1 Purpose

This document defines the software requirements for the *People of the Holocaust* senior design project. It describes the required behavior of the public website, database, data processing pipeline, & testing processes.

## 2.2 Scope

*People of the Holocaust* is a web based historical research tool that provides person centered information from the *Encyclopedia of Camps and Ghettos*. The system will allow users to browse and search for people, places, and map locations. The system will eventually pull information from a database rather than CSV files - which is what it currently does.

The project includes a Python interpreter for processing source text, a database for storing structured records, and a React frontend for displaying said records to users.

## 2.3 Definitions and Acronyms

| Term | Definition |
|---|---|
| ECG | Encyclopedia of Camps and Ghettos |
| Activity | A structured record showing an action or event associated with a person or place. |
| Interpreter | The Python data processing pipeline used to extract records from the ECG |
| Frontend | The React and Typescript web application used to show project data. |
| Database | Persistent data storage containing people, places, activities, & map records. |

# 3 Overall Description

## 3.1 Product Perspective

The system has three main parts: a frontend website, a database, and a backend interpreter. The frontend gives the public a user interface. The database stores structured people, places, activities, and location records. The interpreter processes encyclopedia text and prepares records for storage in the database.

## 3.2 Product Functions

1. Display a public homepage explaining the purpose behind the project.
2. Allow users to search and browse people records.
3. Allow users to search and browse place records.
4. Display historical locations on an interactive map.
5. Store structured data in a database.
6. Process source material into structured records.

## 3.3 User Classes

1. Researchers and academics using the system for historical research.
2. Students using the system for learning and class projects.
3. General public users interested in the history behind the Holocaust.
4. Project maintainers responsible for the frontend, database, and interpreter.

# 4 Functional Requirements

| **ID** | **Requirement** | **Priority** |
|---|---|---|
| FR-1 | The system shall provide a public website for viewing *People of the Holocaust* data. | High |
| FR-2 | The system shall not require users to create an account or log in to view public records. | High |
| FR-3 | The website shall provide a homepage that explains the background and purpose of the project. | High |
| FR-4 | The website shall provide a People page that pull person records from the database. | High |
| FR-5 | The website shall allow users to search people by name. | High |
| FR-6 | The website shall always allow users to select a person and view related details. | High |
| FR-7 | The website shall provide a Places page that pulls place records from the database. | High |
| FR-8 | The website shall allow users to search places by name. | High |

| | | |
|---|---|---|
| FR-9 | The website shall allow users to select a place and view related details. | High |
| FR-10 | The website shall provide an interactive map of historical locations. | High |
| FR-11 | The map shall display markers for places with valid latitude and longitude values. | High |
| FR-12 | The map shall display place information when a marker is selected. | High |
| FR-13 | The interpreter shall clean raw person names into structured person records. | High |
| FR-14 | The interpreter shall retrieve and clean article text from the *ECG*. | High |
| FR-15 | The interpreter shall identify sentences that contain known person names. | High |
| FR-16 | The interpreter shall pull activity records from identified sentences when possible. | High |
| FR-17 | The interpreter shall associate activity records with people and places. | High |
| FR-18 | The interpreter shall write processed records to project database. | High |
| FR-19 | The interpreter shall log article processing logs for later review. | High |

# 5 Non-Functional Requirements

| **ID** | **Requirement** | **Priority** |
|---|---|---|
| NFR-1 | The website shall be usable in modern browsers. | High |
| NFR-2 | The website shall pull public data from the database through API requests or database client. | High |
| NFR-3 | The frontend shall be buildable using the project build command. | High |
| NFR-4 | The interpreter shall run Python 3.11 or newer. | High |
| NFR-5 | The system shall preserve historical meaning of extracted material. | High |
| NFR-6 | The website shall remain usable when optional data fields are missing. | High |
| NFR-7 | Database credentials and service keys shall not be exposed in frontend code or committed to version control. | High |
| NFR-8 | The system shall respect source permissions. | High |
| NFR-9 | The system shall be maintainable by future developers. | High |

# 6 Data Requirements

| Data Item | Description |
|---|---|
| People Records | records containing person identifiers, first names, last names, alternative names, titles, places, organizations, and designations. |
| Place Records | Records containing place identifiers, names, types, subtypes, countries, and location identifiers. |
| Activity Records | Records connecting people, places, actions, and supporting sentence/paragraph details. |
| Map Records | Records containing place names, latitude, longitude, type, subtype, and current country for map display. |
| Article Source Records | Records identifying encyclopedia articles processed by the interpreter. |
| Error Logs | Records of source articles or processing steps that failed during interpreter execution. |

The database will be the primary source for website data. The CSV and JSON files will be used during development, testing, or migration, but they are not the intended long-term data source.

# 7 User Requirements

| ID | User Requirement |
|---|---|
| USR-1 | As a researcher, I want to search for a person by name so that I can find relevant historical records quickly. |
| USR-2 | As a researcher, I want to search for a place so that I can find locations of interest. |
| USR-3 | As a public user, I want to view an interactive map so that I can explore historical places geographically. |
| USR-4 | As a public user, I want to access the website without an account. |
| USR-5 | As a project maintainer, I want the interpreter to identify key sentences automatically so that data creation is more efficient. |
| USR-6 | As a project maintainer, I want failed article processing attempts to be logged so that errors can be reviewed. |

# 8 Constraints and Assumptions

## 8.1 Constraints

1. The frontend is implemented using React, TypeScript, and Vite.

2. The frontend is expected to retrieve structured records from the project database.
3. The interactive map depends on LeafLet and OpenStreetMap tiles.
4. The interpreter is implemented in Python and requires Python 3.11 or later.
5. Database access requires credentials that must not be committed to version control.
6. The system must respect source permissions for encyclopedia content.

### 8.2 Assumptions

1. The project has permission to use the required source material.
2. Database records are populated before frontend deployment.
3. records with missing optional fields should not prevent the website from loading.
4. A sentence may refer to more that one person.
5. Some extracted activity records may be incomplete when parsing cannot confidently identify an action.

## 9 Acceptance Criteria

| ID | Acceptance Criterion |
|---|---|
| AC-1 | The homepage loads and shows the project purpose and description. |
| AC-2 | The People page loads person records from the database. |
| AC-3 | Searching on the People page filters records based on entered text. |
| AC-4 | Selecting a person displays that person's details and relationships. |
| AC-5 | The Places page loads place records from the database. |
| AC-6 | Searching on the Places page filters records based on entered text. |
| AC-7 | Selecting a place shows that place's details and related activities. |
| AC-8 | The map loads location records and shows markers for records with valid coordinates. |
| AC-9 | Marker popups accurately show place information. |
| AC-10 | The frontend build command completes successfully. |
| AC-11 | The interpreter tests complete successfully. |
| AC-12 | The interpreter logs failed processing attempts without stopping the full processing run. |

## 10 Verification Plan

The requirements will be verified using build checks, manual website testing, and Python unit tests for the interpreter.

1. Run the frontend build process.
2. Manually verify homepage, navigation, People page, Places page, and map behavior.

3. Verify that people, place, activity, and map records correctly load from the database.
4. Verify that search inputs filter people and places correctly.
5. Verify that selected records display associated activities.
6. Run interpreter tests using `pytest`.
7. Review the interpreter error logs after processing articles.